

CO2-AT-1

Name: Monica Sentam91.K.

Analytical

Reg no: 192524315

Questions-

Course: CSA0311 -

Data Structures.

1.

Circular Queue Operations:

Concept overview: A circular Queue is a linear data structure that follows the FIFO (First In, First out) principle, but connects the last position back to the first position to utilise space.

- Size (N): 5 (indices: 0, 1, 2, 3, 4)
- Index Formulae (using modulo arithmetic:)
 - $Front = (Front + 1) \% N$ (during Dequeue)
 - $Rear = (Rear + 1) \% N$ (during Enqueue)
- Initial State: $FRONT = -1$, $REAR = -1$ (Empty queue.)

Step by step execution.

STEP 0.

operation: Initial state

Array state [0, 1, 2, 3, 4]: [-, -, -, -, -]

Front: -1 Rear: -1

Explanation: Queue is Empty.

STEP 1

operation: Enqueue (20)

Array state: [20, -, -, -, -]

Front: 0

Rear: 0

Explanation: First element Inserted. FRONT and REAR set to 0.

STEP 2

operation: Enqueue (30)

Array State: [20, 30, -, -, -]

Front: 0

Rear: 1

Explanation: $Rear = (0 + 1) \% 5 = 1$

Insert 30 at index 1.

Step	operation	Array	FRONT	REAR	EXPLANATION
3	Enqueue(10)	[20, 30, 10, -]	0	2	$REAR = (1+1) \% 5 = 2$ Insert 10 at Index 2.
4	Dequeue()	[-, 30, 10, -]	1	2	Element 20 Removed
5	Enqueue(80)	[-, 30, 10, 80, -]	1	3	Insert 80 at Index 3.
6	Dequeue()	[-, -, 10, 80, -]	2	3	Element 30 Removed.
7	Enqueue(90)	[-, -, 10, 80, 90]	2	4	Insert 90 at Index 4.
8	Enqueue(100)	[100, -, 10, 80, 90]	2	0	$REAR = (4+1) \% 5 = 0$ Insert 100 at Index 0.
9	Dequeue()	[100, -, -, 80, 90]	3	0	Element 30 Removed $FRONT = (2+1) \% 5 = 3$
10	Dequeue()	[100, -, -, -, 90]	4	0	Element 80 Removed $FRONT = 4$.
11	Enqueue(60)	[100, 60, -, -, 90]	4	1	$REAR = (0+1) \% 5 = 1$ Insert 60 at Index 1.
12	Enqueue(90)	[100, 60, 90, -, 90]	4	2	$REAR = (1+1) \% 5 = 2$ Insert 90 at Index 2.

Final Result:

- Final Queue Array: [100, 60, 90, -, 90]
- Front Element: 90 at position/index 4.
- Rear Element: 90 at position/index 2.

Note on Circular Queue Efficiency: Notice how index 0, 1, and 2 were reused after deletions. In a standard linear queue, these positions would be wasted.

Binary Search Algorithm:

Concept overview: Binary search works on a sorted array by repeatedly dividing the search interval in half.

• Formulae:

$$\text{mid} = \left\lceil \frac{\text{low} + \text{high}}{2} \right\rceil$$

• Given Array (N=10).

Index	value.
0	66
1	111
2	212
3	323
4	432
5	543
6	643
7	649
8	777
9	888

case: 1 key Element = 111

• Iteration 1:

• low = 0, high = 9

• $\text{mid} = \left\lceil \frac{(0+9)}{2} \right\rceil = 4$

• $\text{Array}[4] = 432$

• Compare: $111 < 432$

Search left half (high = mid - 1 = 3)

Iteration: 2

- $low = 0$, $high = 3$
- $mid = \lceil (0+3)/2 \rceil = 1$
- $Array[1] = 111$
- Compare: $111 == 111 \Rightarrow$ Match found.
- Result: Key 111 found at index 1 (00 and position)

Case: 2 Key element = 888

Iteration: 1

- $low = 0$, $high = 9$
- $mid = \lceil (0+9)/2 \rceil = 4$
- $Array[4] = 432$
- Compare: $888 > 432 \Rightarrow$ Search right half
($low = mid + 1 = 5$)

Iteration: 2

- $low = 5$, $high = 9$
- $mid = \lceil (5+9)/2 \rceil = 7$
- $Array[7] = 649$
- Compare: $888 > 649 \Rightarrow$ Search right half
($low = mid + 1 = 8$)

Iteration: 3.

- $low = 8$, $high = 9$
- $mid = \lceil (8+9)/2 \rceil = 8$
- $Array[8] = 777$
- Compare: $888 > 777 \Rightarrow$ Search right half ($low = mid + 1 = 9$).

Iteration: 4

- $low = 9$, $high = 9$
- $mid = \lceil (9+9)/2 \rceil = 9$
- $Array[9] = 888$
- Compare: $888 == 888 \Rightarrow$ Match found.
- Result: key 888 found at index: 9

ASUS Vivobook

Case: 3 Key element = 999.

Iteration: 1

- low = 0, high = 9

- mid = 4 $\Rightarrow$ Array[4] = 482

- $999 > 482 \Rightarrow$ low = 5.

Iteration 2.

- low = 5, high = 9

- mid = 7 $\Rightarrow$ Array[7] = 649

- $999 > 649 \Rightarrow$ low = 8.

Iteration 3.

- low = 8, high = 9

- mid = 8 $\Rightarrow$ Array[8] = 777

- $999 > 777 \Rightarrow$ low = 9.

Iteration 4.

- low = 9, high = 9

- mid = 9 $\Rightarrow$ Array[9] = 888

- $999 > 888 \Rightarrow$ low = 10.

Iteration 5.

- low = 10, high = 9

- Loop condition fails (low > high)

- Result: key 999 Not Found in the array.

Time and Space Complexity:

- Time complexity:

- Base case: $O(1)$ (when the target element is at the middle position in the first step).

- Worst/Average case: $O(\log_2 N)$ - Here, $N=10$, so at most $\approx \lceil \log_2(10) \rceil = 4$ comparisons are needed.

- Space complexity:

- Iterative Approach: $O(1)$ (Auxiliary Space)

- Recursive Approach: $O(\log N)$ (call stack space).